EvacX: Intelligent Dynamic Evacuation & Emergency Response System

1. Introduction

During emergencies such as fires, earthquakes or gas leaks, people often do not know the safest way out, and fixed evacuation routes can become unsafe. EvacX calculates the safest route dynamically, based on where the emergency is.

2. Objectives

- Represent a building as a digital map with exits
- Mark blocked or dangerous areas
- Compute the safest route automatically
- Send real-time alerts to all users
- Provide an admin panel, incident history and analytics

3. Key Features

- Building modelled as a graph (rooms, halls, stairs, exits)
- Dijkstra's algorithm with a crowd-density penalty; blocked areas are excluded
- Nearest safe exit recommendation
- Real-time alerts using WebSockets
- Admin panel (password protected) to report and clear emergency zones
- Simulated crowd-density sensors
- Incident history and basic analytics

4. Technologies Used

Layer	Technology
Backend	Go (net/http standard library)
Real-time	WebSockets (gorilla/websocket)
Storage	JSON file (incidents.json)
Frontend	HTML, CSS, JavaScript, SVG
Algorithm	Dijkstra with container/heap
Testing	Go testing package

5. Implementation

1. **Graph model:** each location is a node and each corridor is an edge with a distance. Route cost = distance + 0.5 x number of people at the next location. 2. **Routing:** Dijkstra's algorithm starts from the user's location and returns the first (cheapest) exit it reaches. Blocked nodes are never visited. 3. **Emergency handling:** when the admin marks an area dangerous, the node is blocked, the incident is saved, and a WebSocket alert is broadcast. Every

user's page recomputes its route instantly. 4. **Simulation:** crowd sensors and fire detection are simulated (a random crowd generator and the admin button). Real sensors could use the same API. 5. **Testing:** unit tests check the normal route, rerouting when a node is blocked, and the case where no safe route exists.

6. Screenshots

Normal route

Crowd density

Admin panel

Real-time alert

Rerouted path after blocking Main Stairs

Incident history and analytics

7. How to Run

1. Install Go
2. `git clone https://github.com/JapneetKaurAI/evacx.git`
3. `cd evacx`
4. `go mod tidy`
5. `go run ./cmd/server`
6. Open http://localhost:8080 (admin page: /admin.html, password: admin123)

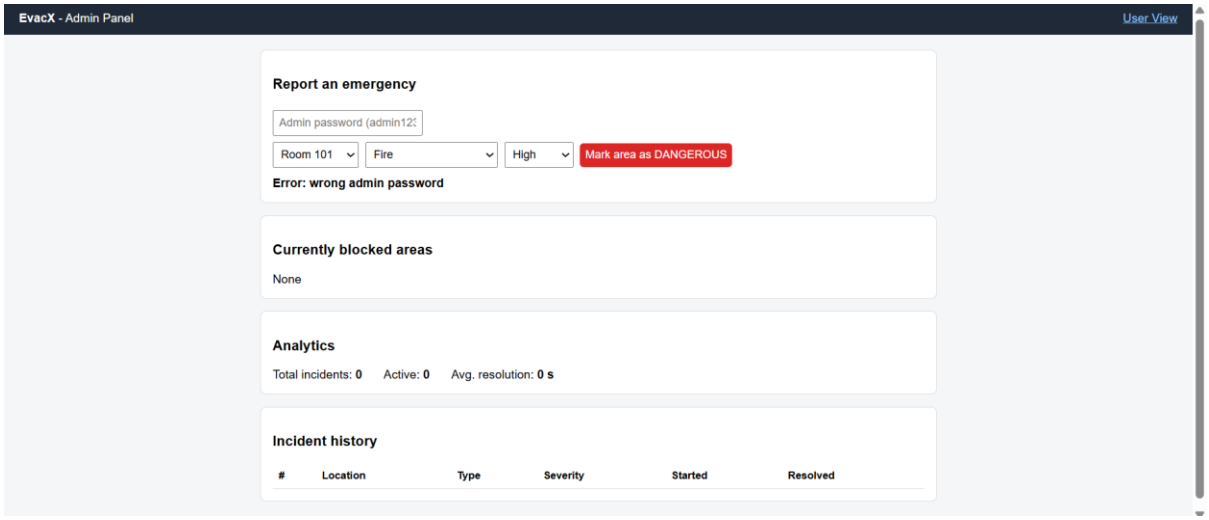
8. Limitations and Future Work

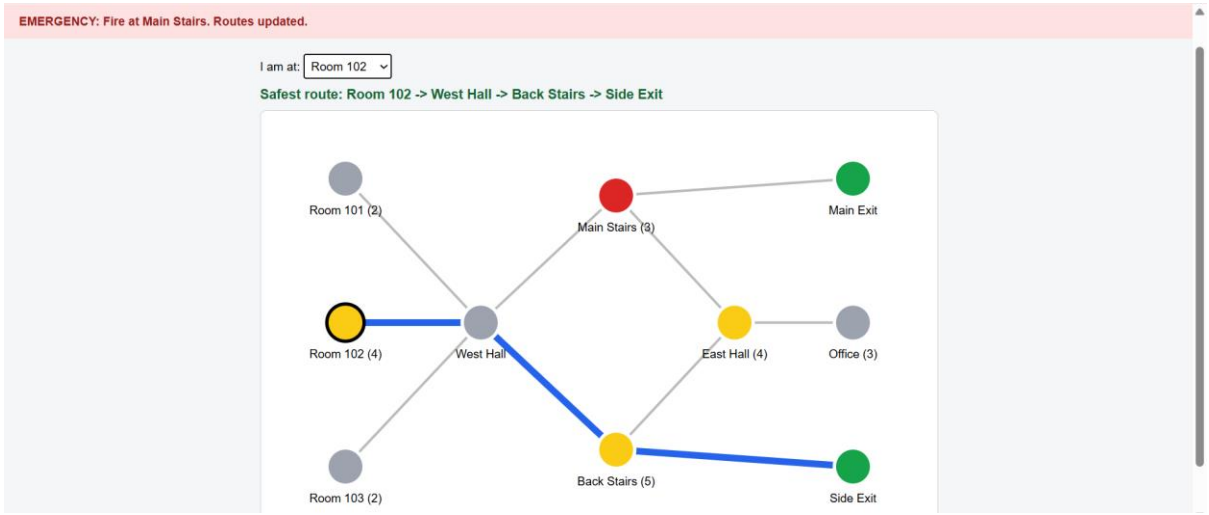
- Sensor input and indoor positioning are simulated
- Future: BLE/Wi-Fi indoor positioning, mobile app, multi-floor and multi-building support, SQL database

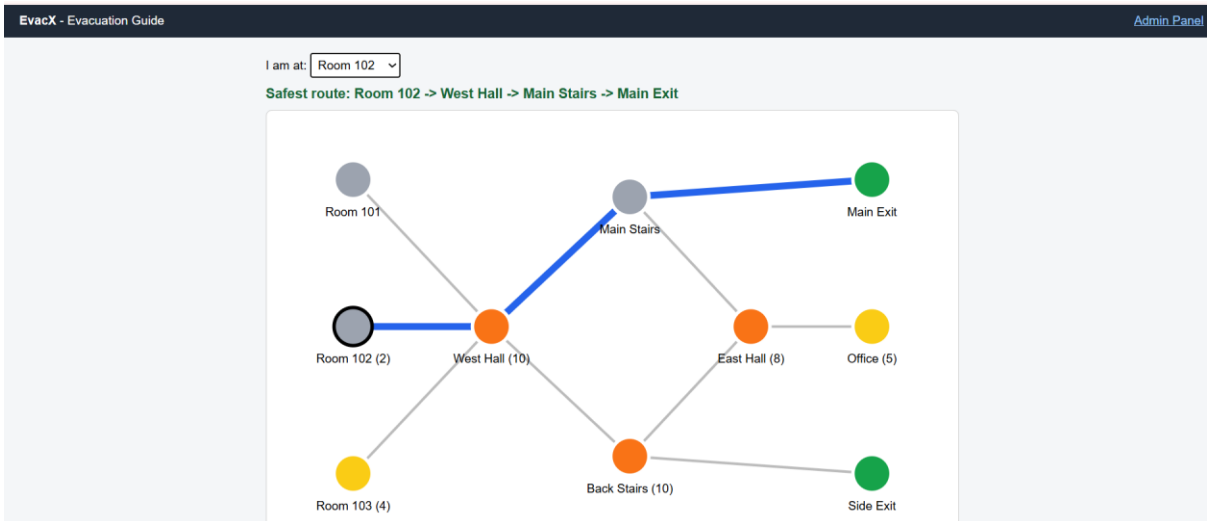
9. Author

Japneet kaur, 2401720015, Bsc(Hons) Computer Science

10.Screenshots







Report an emergency

••••••••

Main Stairs | Fire | High | Mark area as DANGEROUS

Zone marked dangerous. Alerts sent.

Currently blocked areas

Main Stairs Mark as safe

Analytics

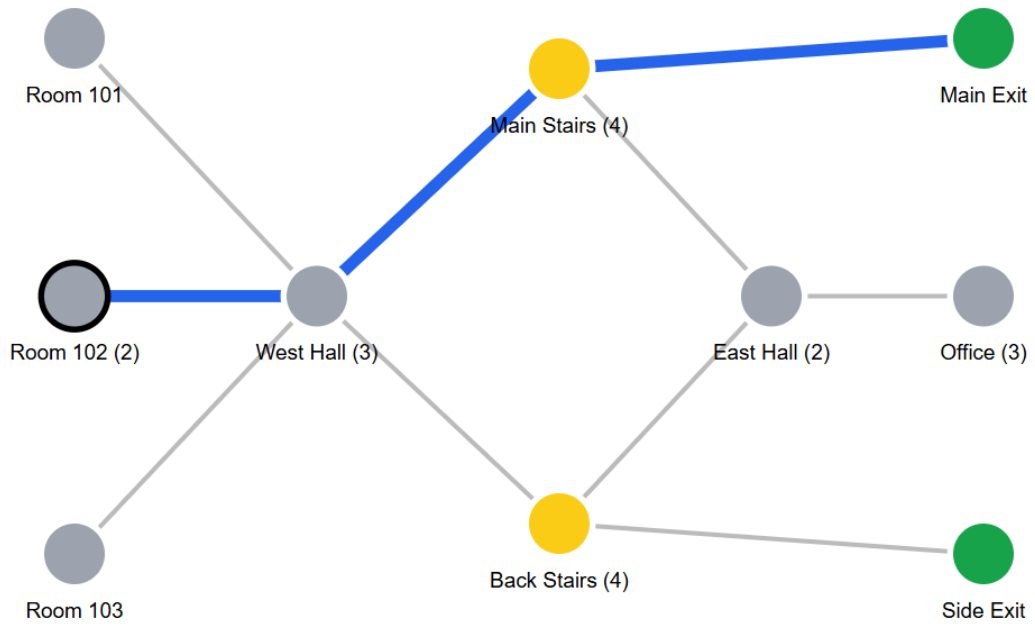
Total incidents: **1** Active: **1** Avg. resolution: **0 s**

Incident history

#	Location	Type	Severity	Started	Resolved
1	Main Stairs	Fire	High	11/6/2026, 11:49:08 AM	ACTIVE

I am at: Room 102

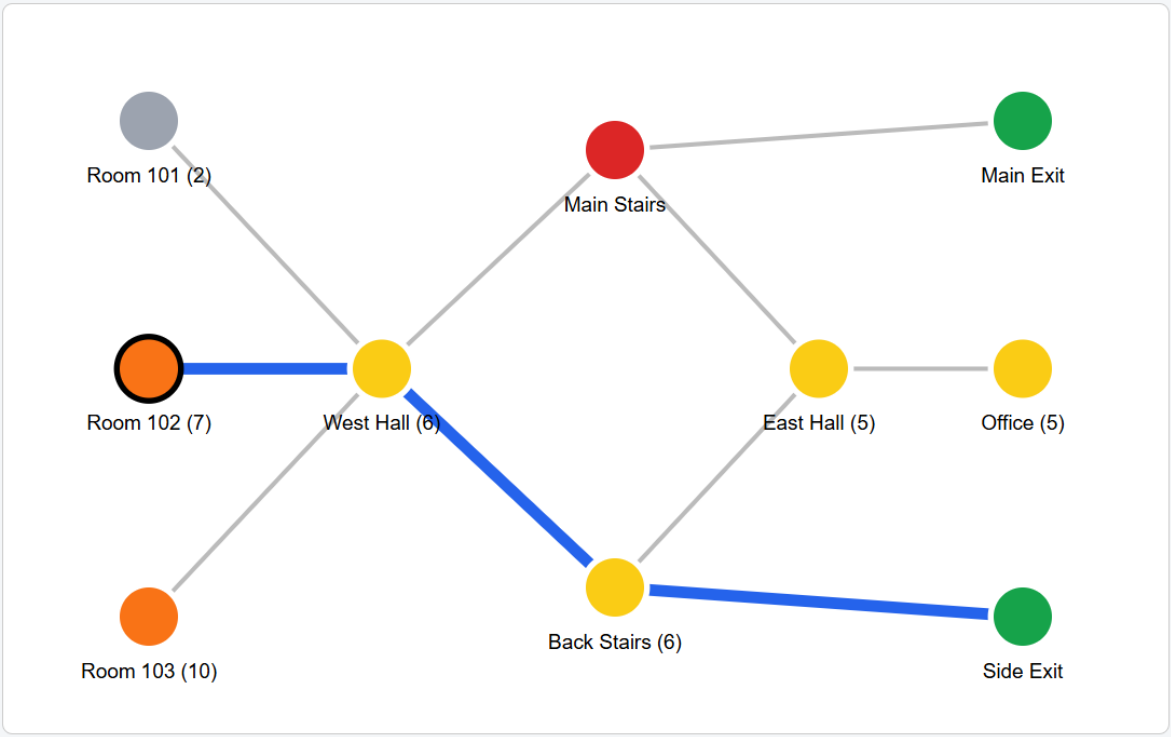
Safest route: Room 102 -> West Hall -> Main Stairs -> Main Exit



Blue line = safest route Green = Exit Red = Dangerous / blocked Yellow = Moderate crowd Orange = Heavy crowd

I am at: Room 102

Safest route: Room 102 -> West Hall -> Back Stairs -> Side Exit



Blue line = safest route Green = Exit Red = Dangerous / blocked Yellow = Moderate crowd Orange = Heavy crowd

Report an emergency

••••••••

Main Stairs | Fire | High | Mark area as DANGEROUS

Zone marked dangerous. Alerts sent.

Currently blocked areas

Main Stairs Mark as safe

Analytics

Total incidents: **1** Active: **1** Avg. resolution: **0 s**

Incident history

#	Location	Type	Severity	Started	Resolved